

Pertinent Thoughts — Scout (530 College Basketball Pipeline)

This document mirrors **Pertinent_Thoughts.md** (Army / OER work): important discoveries, reflections on code and results, open problems, and directions worth investigating for the **Scout** dissertation thread — ESPN box → SR advanced → player-season panel → leave-one-out teammate pool quality → draft outcome EDA.

How to use it: Add dated entries or new `##` sections as you go. Each block can follow the template: **Topic** → **Content to consider including** → **Potential placement** → **Key points** (plus optional implementation notes).

SR-ESPN School Slugs, Small Colleges, and Coverage Equity

Topic: Unmatched schools after `bpm_merge.run_match`; manual crosswalk / alias loop; re-scrape behavior.

Content to consider including:

Many unmatched player-season rows cluster on **small religious colleges, HBCUs, NAIA-style programs, and renamed schools** where the heuristic `school_slug` (from `team_short_display_name`) does not match Sports Reference's URL segment. The pipeline already surfaces this in `bpm_panel_rows_unmatched.csv` and optional `print_unmatched_school_lists` (aggregated by `team_id` / slug). Fixing mappings lives in `DO_NOT_ERASE/sr_school_slug_crosswalk.csv` (canonical `team_id` → `school_slug`) and `DO_NOT_ERASE/sr_school_slug_aliases.csv` (`panel_slug` → `sr_slug` for URL resolution while keeping merge keys consistent). After edits, SR refresh with `sr_refresh_do_scrape=True` only fetches **missing** `(school_slug, sr_year)` pairs in raw (resume logic), not a full re-scrape. **403** pairs are skipped via `bpm_scrape_skip_pairs.csv` until removed.

Potential placement:

- Data / linkage section (ESPN vs SR)
- Limitations (non-random missing SR advanced by institution type)
- Future work: conference- or success-based imputation of teammate quality where SR is absent

Key points:

- Unmatched SR is often institutional, not random — affects inference if draft-relevant talent concentrates in thin coverage cells.
 - Crosswalk vs alias: crosswalk changes the canonical slug; alias fixes the URL when the panel slug should stay stable.
 - Re-scrape is incremental by design; renaming a slug creates **new** job keys — old raw rows under the old slug do not automatically attach.
-

Ventile EDA vs Survival / CIF Framing (Officers Comparison)

Topic: What the inverted-U / binned draft-rate plots are and are not.

Content to consider including:

The 530 ventile figure bins **cross-sectional** `poolq_loo` (leave-one-out mean teammate `perf` within team-season) and plots **mean** `Y_draft` per bin vs mean `poolq_loo` — empirical draft **rate**, not time-to-event. Optional overlay is **quadratic LPM** in

`poolq_loo` on the micro rows. This is **not** a cumulative incidence or survival curve; comparison to the officers workflow (CIF by bin, then bar chart of terminal values) is a **design choice** distinction worth one clear paragraph in methods. `poolq_binning` supports `quantile` (~equal n per bin; “ventiles” when `ventiles=20`) and `equal_width` on `poolq_loo`, with export slugs disambiguating runs.

Potential placement:

- Methods (nonparametric binning vs survival)
- Discussion when contrasting Scout to Army analyses
- Limitations (no time path to draft within season-year structure)

Key points:

- Each row is player–season–team; `poolq_loo` is within that season’s roster context.
- Y-axis is bin-level **draft rate**, not CIF at a horizon.
- Binning mode and z-scoring `perf` within season change interpretation of `poolq_loo` units.

Performance Measures: Box vs SR and LOO Interpretation

Topic: What enters `perf` and teammate pool quality; minutes / PPM sources.

Content to consider including:

`minutes`, `points`, `ppm`, `games` come from **ESPN game-level box** aggregated to season-team totals (PPM = total points / total minutes). **SR raw** (`bpm_player_season_raw.csv`) is the **advanced** table: `MP`, `G`, `GS`, BPM family, PER, WS, etc. — **no SR-derived PPM** in this scrape. Panel merge exposes `mp_sr` separately from box `minutes`. `perf_metric` and optional `perf_zscore_within_season` determine what is copied into `perf` before LOO; `poolq_loo` is always defined from teammates’ `perf`, not from a career average across seasons.

Potential placement:

- Variable construction / data integration
- Robustness (re-run key specs across `perf_metric` choices; filenames already slugged)

Key points:

- Same athlete can contribute multiple rows (multiple seasons / teams); LOO is **within** team-season.
- SR coverage gaps leave `perf` missing for SR-backed metrics unless merged.

High School Performance, Teammate Pools, and Binning (Advisor Discussion)

Topic: Enrich the panel with HS data; bin on pre-college ability while preserving leave-one-out **team** structure.

Content to consider including:

Discussion with advisor: add **high school** (or recruiting) data so players are **binned** along a dimension defined by **HS performance metrics** (e.g. composite rank, scouting grades, stats), rather than binning on `poolq_loo` built from **collegiate seasonxteam** `perf` as in the current 530 ventile EDA. The conceptual **team pool** should **remain**: still compute **leave-one-out**

teammate pool quality within `(team_id, season)`, but drive `perf` (or a parallel HS column) from **high-school-level measures** so “how good are my teammates *as projected from HS?*” replaces or complements “how good are my teammates *this college season on this stat?*” Implementation requires a **merge** to `athlete_id` (or a stable recruiting ID), decisions on **missing HS** coverage, and whether HS metrics are **z-scored within cohort** (class year, position) before LOO. The ventile / LPM machinery can stay the same mechanically once `poolq_loo` is defined from the chosen HS-based `perf`.

Potential placement:

- Methods (covariate timing: all pre-college information for the binning axis)
- Data section (HS / recruiting sources, linkage rate)
- Discussion (interpretation: sorting on ex-ante talent vs realized college performance)
- Limitations (selection into observable HS data; international / JUCO paths)

Key points:

- Separates **who you are surrounded by in HS terms** from **college box/SR realization** for stratification.
- LOO algebra unchanged: still exclude self within team-season; only the input to `perf` changes.
- Creates a clear story for “peer pool at arrival” vs current “peer pool on realized college metric.”
- Not yet in `sports_pipeline` — needs schema, merge QA, and sensitivity to missing HS.

Restricting to Teams With at Least One Drafted Player

Topic: Drop rows for teams with **no** drafted player under a clear rule (program vs season).

Content to consider including:

Implemented in code (`panel_build.filter_panel`): `restrict_teams_by_draftees` (default **True**) and `draftee_restriction`: `all_time` — keep only `team_id` with ≥ 1 row with `Y_draft==1` anywhere in the filtered sample before this step; `season` — keep only `(team_id, season)` where ≥ 1 roster member has `Y_draft==1` that season. Applied after `dropna(poolq_loo, Y_draft)` and `min_minutes`, so ventiles, LPM, and integrity `use` all align. Export slugs gain `_tdalltime` or `_tdseason` when the restriction is on (see `perf_metric.export_plot_slug`). Motivation: focus draft-rate vs pool-quality on environments where draft outcomes are not structurally zero on the roster. Trade-offs: smaller sample, selection (e.g. low-majors), changed interpretation — report **counts dropped** and run **robustness** with `restrict_teams_by_draftees=False`.

Potential placement:

- Sample definition / inclusion rules
- Robustness appendix (full sample vs draft-exposed-teams-only)
- Limitations (generalization beyond teams that ever produce a draft pick)

Key points:

- Sharpens comparison sets where draft outcome is empirically possible on the roster.
 - Risks truncating heterogeneity — justify and show robustness.
 - `all_time` vs `season` answers “program ever had a draftee in window” vs “this season’s roster had a draftee.”
-

Sorting Noise Robustness Thought

Topic: Later robustness version for noisy assortative sorting in `537_Sports_Simulation.ipynb`.

Content to consider including:

For the first noisy-sorting implementation, use raw Gaussian sorting noise:

`sorting_signal_i = A_i + epsilon_i`, where `epsilon_i ~ Normal(0, SORTING_NOISE_SD)`.

This keeps the first version easy to explain: true ability `A_i` is unchanged, but the pool assignment process observes a noisy version of ability. When `SORTING_NOISE_SD = 0`, sorting is perfectly assortative; as it increases, pool assignment becomes less perfectly ordered.

Later, revisit a relative-noise version:

`epsilon_sd = SORTING_NOISE_FRACTION * sd(A_i)`.

That version may be more portable across different ability distributions because the noise scale adjusts to the observed spread of ability. Hold this for a robustness/sensitivity branch after the raw-noise version is clear.

Potential placement:

- Simulation appendix or robustness subsection
- Notes around noisy sorting in `537_Sports_Simulation.ipynb`

Key points:

- Noise is **only for sorting into pools**, not for true ability or promotion weights.
- Raw `SORTING_NOISE_SD` is easier to teach first.
- Relative `SORTING_NOISE_FRACTION * sd(A_i)` is the later robustness idea to remember.

Small Things to Check Later

- Confirm whether `Y_draft` should be “ever drafted” vs draft **in or after** season k for causal timing stories.
- Document any manual edits to `sr_school_slug_crosswalk.csv` in a one-line changelog row or git commit message when possible.
- `export_panel_after_run` during multi-metric sweeps overwrites one path — document if a dated archive is needed.
- Implement and document **HS → athlete** merge keys; add `poolq_loo_hs` (or swap `perf` source) when data land.
- Log **N excluded** by `restrict_teams_by_draftees` in a one-off QA cell or export (optional enhancement).